```
/* ============================================================
FUNCTIONS IN ORACLE PL/SQL

   Topics:
   - Function Syntax
   - Return Data Types
   - Calling Functions
   - Functions in SQL Statements
   - Deterministic Functions

   Labs:
   - Grade Function
   - Tax Calculation Function
   - Customer Loyalty Function
   ============================================================ */

SET SERVEROUTPUT ON SIZE UNLIMITED;

/* ============================================================
   1. DROP OLD TABLES
   ============================================================ */

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE students';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE employees';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE customers';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

/* ============================================================
   2. CREATE TABLES
   ============================================================ */

CREATE TABLE students
(
    student_id   NUMBER PRIMARY KEY,
    student_name VARCHAR2(100),
    mark         NUMBER
);
```

```
CREATE TABLE employees
(
    employee_id   NUMBER PRIMARY KEY,
    employee_name VARCHAR2(100),
    salary        NUMBER,
    department    VARCHAR2(50)
);

CREATE TABLE customers
(
    customer_id     NUMBER PRIMARY KEY,
    customer_name   VARCHAR2(100),
    total_purchase  NUMBER,
    status          VARCHAR2(20)
);

/* ========================================================
   3. INSERT SAMPLE DATA
   ======================================================== */

INSERT INTO students VALUES (1, 'Aung Aung', 85);
INSERT INTO students VALUES (2, 'Mg Mg', 72);
INSERT INTO students VALUES (3, 'Su Su', 55);
INSERT INTO students VALUES (4, 'Hla Hla', 35);

INSERT INTO employees VALUES (1, 'Kyaw Kyaw', 900000, 'IT');
INSERT INTO employees VALUES (2, 'Thida', 1500000, 'Finance');
INSERT INTO employees VALUES (3, 'Mya Mya', 500000, 'HR');

INSERT INTO customers VALUES (1, 'ABC Trading', 5000000, 'ACTIVE');
INSERT INTO customers VALUES (2, 'Global Tech', 2000000, 'ACTIVE');
INSERT INTO customers VALUES (3, 'Myanmar Retail', 500000, 'ACTIVE');

COMMIT;

/* ========================================================
   4. FUNCTION SYNTAX
   Basic syntax:

   CREATE OR REPLACE FUNCTION function_name
   (
       parameter_name IN data_type
   )
   RETURN return_data_type
   AS
       local_variable data_type;
   BEGIN
       statements;
       RETURN value;
   EXCEPTION
       exception_handling;
   END;
   /
   ======================================================== */
```

```
/* ============================================================
   5. SIMPLE FUNCTION RETURNING VARCHAR2
   ============================================================ */

CREATE OR REPLACE FUNCTION get_welcome_message
RETURN VARCHAR2
AS
BEGIN
    RETURN 'Welcome to Oracle PL/SQL Functions';
END;
/

/* Calling function from PL/SQL block */
```

```
SQL> BEGIN
    DBMS_OUTPUT.PUT_LINE(get_welcome_message);
END;
/
  2    3    4  Welcome to Oracle PL/SQL Functions

PL/SQL procedure successfully completed.
```

```
/* Calling function from SQL statement */
```

```
SQL> SELECT get_welcome_message AS message FROM dual;

MESSAGE
--------------------------------------------------
Welcome to Oracle PL/SQL Functions
```

```
/* ============================================================
   6. FUNCTION RETURNING NUMBER
   ============================================================ */

CREATE OR REPLACE FUNCTION add_two_numbers
(
    p_num1 IN NUMBER,
    p_num2 IN NUMBER
)
RETURN NUMBER
AS
    v_result NUMBER;
BEGIN
    v_result := p_num1 + p_num2;

    RETURN v_result;
END;
/
```

```
SQL> BEGIN
    DBMS_OUTPUT.PUT_LINE('Result: ' || add_two_numbers(100, 200));
END;
/
  2    3    4  Result: 300

PL/SQL procedure successfully completed.
SQL> SELECT add_two_numbers(500, 300) AS total FROM dual;

     TOTAL
----------
       800
```

```
/* ============================================================
   7. FUNCTION RETURNING DATE
   ============================================================ */

CREATE OR REPLACE FUNCTION get_today_date
RETURN DATE
AS
BEGIN
    RETURN SYSDATE;
END;
/
```

```
SQL> SELECT get_today_date AS today_date FROM dual;

TODAY_DAT
---------
21-JUN-26
```

```
/* ============================================================
   8. FUNCTION WITH LOCAL VARIABLES
   ============================================================ */

CREATE OR REPLACE FUNCTION calculate_bonus
(
    p_salary IN NUMBER
)
RETURN NUMBER
AS
    v_bonus_rate NUMBER;
    v_bonus      NUMBER;
BEGIN
    IF p_salary >= 1000000 THEN
        v_bonus_rate := 0.15;
    ELSE
        v_bonus_rate := 0.10;
    END IF;

    v_bonus := p_salary * v_bonus_rate;

    RETURN v_bonus;
END;
/
```

```
SQL> BEGIN
    DBMS_OUTPUT.PUT_LINE('Bonus: ' || calculate_bonus(1200000));
END;
/
  2    3    4  Bonus: 180000

PL/SQL procedure successfully completed.
```

/* Function used inside SQL SELECT */

```
SQL> SELECT
    employee_id,
    employee_name,
    salary,
    calculate_bonus(salary) AS bonus
FROM employees;
  2    3    4    5    6
EMPLOYEE_ID
-----------
EMPLOYEE_NAME
------------------------------------
    SALARY      BONUS
---------- ----------
         1
Kyaw Kyaw
    900000      90000

         2
Thida
   1500000     225000

EMPLOYEE_ID
-----------
EMPLOYEE_NAME
------------------------------------
    SALARY      BONUS
---------- ----------

         3
Mya Mya
    500000      50000
```

```
/* ============================================================
   9. DETERMINISTIC FUNCTION

   A deterministic function returns the same result
   whenever the same input values are provided.

   Example:
   Tax calculation based only on salary.
   ============================================================ */

CREATE OR REPLACE FUNCTION calculate_fixed_tax
(
    p_salary IN NUMBER
)
RETURN NUMBER
DETERMINISTIC
AS
BEGIN
    RETURN p_salary * 0.05;
END;
/
```

```
SQL> SELECT
    employee_name,
    salary,
    calculate_fixed_tax(salary) AS fixed_tax
FROM employees;
  2    3    4    5
EMPLOYEE_NAME
----------------------------------------
    SALARY  FIXED_TAX
---------- ----------
Kyaw Kyaw
    900000      45000

Thida
   1500000      75000

Mya Mya
    500000      25000
```

```
/* ============================================================
   LAB 1: GRADE FUNCTION

   Requirement:
   - 80 and above = A
   - 60 to 79 = B
   - 40 to 59 = C
   - Below 40 = Fail
   ============================================================ */

CREATE OR REPLACE FUNCTION get_grade
(
    p_mark IN NUMBER
)
RETURN VARCHAR2
AS
    v_grade VARCHAR2(10);
BEGIN
    IF p_mark >= 80 THEN
        v_grade := 'A';
    ELSIF p_mark >= 60 THEN
        v_grade := 'B';
    ELSIF p_mark >= 40 THEN
        v_grade := 'C';
    ELSE
        v_grade := 'Fail';
    END IF;

    RETURN v_grade;
END;
/

/* Calling Grade Function */
```

```
SQL> BEGIN
    DBMS_OUTPUT.PUT_LINE('Grade: ' || get_grade(85));
    DBMS_OUTPUT.PUT_LINE('Grade: ' || get_grade(55));
END;
/
  2    3    4    5  Grade: A
Grade: C

PL/SQL procedure successfully completed.
```

/* Grade Function in SQL Statement */

```
SQL> SELECT
    student_id,
    student_name,
    mark,
    get_grade(mark) AS grade
FROM students;
  2    3    4    5    6
STUDENT_ID
----------
STUDENT_NAME
------------------------------
      MARK
----------
GRADE
------------------------------
         1
Aung Aung
        85
A


STUDENT_ID
----------
STUDENT_NAME
------------------------------
      MARK
----------
GRADE
------------------------------
         2
Mg Mg
        72
B
```

```
/* ============================================================
   LAB 2: TAX CALCULATION FUNCTION

   Requirement:
   - Salary >= 1,000,000 => Tax 10%
   - Salary >= 500,000   => Tax 5%
   - Salary below 500,000 => Tax 2%
   ============================================================ */

CREATE OR REPLACE FUNCTION calculate_tax
(
    p_salary IN NUMBER
)
RETURN NUMBER
AS
    v_tax_rate NUMBER;
    v_tax      NUMBER;
BEGIN
    IF p_salary >= 1000000 THEN
        v_tax_rate := 0.10;
    ELSIF p_salary >= 500000 THEN
        v_tax_rate := 0.05;
    ELSE
        v_tax_rate := 0.02;
    END IF;

    v_tax := p_salary * v_tax_rate;

    RETURN v_tax;
END;
/

/* Calling Tax Function */
```

```
SQL> BEGIN
    DBMS_OUTPUT.PUT_LINE('Tax Amount: ' || calculate_tax(1500000));
    DBMS_OUTPUT.PUT_LINE('Tax Amount: ' || calculate_tax(700000));
END;
/
  2    3    4    5  Tax Amount: 150000
Tax Amount: 35000

PL/SQL procedure successfully completed.
```

/* Tax Function in SQL Statement */

```
SQL> SELECT
    employee_id,
    employee_name,
    salary,
    calculate_tax(salary) AS tax_amount,
    salary - calculate_tax(salary) AS net_salary
FROM employees;
  2    3    4    5    6    7
EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------
    SALARY TAX_AMOUNT NET_SALARY
---------- ---------- ----------
         1
Kyaw Kyaw
    900000      45000     855000

         2
Thida
   1500000     150000    1350000

EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------
    SALARY TAX_AMOUNT NET_SALARY
---------- ---------- ----------

         3
Mya Mya
    500000      25000     475000
```

```
/* ============================================================
   LAB 3: CUSTOMER LOYALTY FUNCTION

   Requirement:
   - Purchase >= 5,000,000 => PLATINUM
   - Purchase >= 2,000,000 => GOLD
   - Purchase >= 1,000,000 => SILVER
   - Below 1,000,000       => BASIC
   ============================================================ */

CREATE OR REPLACE FUNCTION get_customer_loyalty
(
    p_total_purchase IN NUMBER
)
RETURN VARCHAR2
AS
    v_loyalty_level VARCHAR2(20);
BEGIN
    IF p_total_purchase >= 5000000 THEN
        v_loyalty_level := 'PLATINUM';
```

```
    ELSIF p_total_purchase >= 2000000 THEN
        v_loyalty_level := 'GOLD';
    ELSIF p_total_purchase >= 1000000 THEN
        v_loyalty_level := 'SILVER';
    ELSE
        v_loyalty_level := 'BASIC';
    END IF;

    RETURN v_loyalty_level;
END;
/

/* Calling Customer Loyalty Function */
```

```
SQL> BEGIN
    DBMS_OUTPUT.PUT_LINE('Loyalty Level: ' || get_customer_loyalty(5000000));
    DBMS_OUTPUT.PUT_LINE('Loyalty Level: ' || get_customer_loyalty(800000));
END;
/
  2    3    4    5  Loyalty Level: PLATINUM
Loyalty Level: BASIC

PL/SQL procedure successfully completed.
```

/* Customer Loyalty Function in SQL Statement */

```
SQL> SELECT
    customer_id,
    customer_name,
    total_purchase,
    get_customer_loyalty(total_purchase) AS loyalty_level
FROM customers;
  2    3    4    5    6
CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------
TOTAL_PURCHASE
--------------
LOYALTY_LEVEL
--------------------------------------------------------------
         1
ABC Trading
       5000000
PLATINUM


CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------
TOTAL_PURCHASE
--------------
LOYALTY_LEVEL
--------------------------------------------------------------
         2
Global Tech
       2000000
GOLD
```

```
/* ============================================================
   FINAL CHECKING
   ============================================================ */
```

```
SQL> SELECT * FROM students ORDER BY student_id;

STUDENT_ID
----------
STUDENT_NAME
--------------------------------------------------
      MARK
----------
         1
Aung Aung
        85

         2
Mg Mg
        72

STUDENT_ID
----------
STUDENT_NAME
--------------------------------------------------
      MARK
----------

         3
Su Su
        55

         4
Hla Hla

STUDENT_ID
----------
STUDENT_NAME
--------------------------------------------------
      MARK
----------
        35
```

```
SQL> SELECT * FROM employees ORDER BY employee_id;

EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------
    SALARY DEPARTMENT
---------- ----------------------------------------
         1
Kyaw Kyaw
    900000 IT

         2
Thida
   1500000 Finance

EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------
    SALARY DEPARTMENT
---------- ----------------------------------------

         3
Mya Mya
    500000 HR
```

```
SQL> SELECT * FROM customers ORDER BY customer_id;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------
TOTAL_PURCHASE STATUS
-------------- --------------------
         1
ABC Trading
       5000000 ACTIVE

         2
Global Tech
       2000000 ACTIVE

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------
TOTAL_PURCHASE STATUS
-------------- --------------------

         3
Myanmar Retail
        500000 ACTIVE
```